

Senior Cloud & Production Engineering

A Decision-Making Field Manual - from one VPS to sandbox fleets and ChatGPT-scale systems

Prepared for Anup

5 October 2026

Purpose. This is not an AWS-service dictionary. It is a set of mental models, architectures, deployment patterns, failure models, and projects designed to make you reason like a senior production engineer. The goal is that when a new system appears, you can identify its primitives, estimate its load, choose an operationally sane architecture, and explain exactly why each tool exists.

Source discipline. The Maersk-style IoT example, the iLovePDF-style document platform, the Replit/E2B-style runtime, and the ChatGPT-style architecture are teaching/reference architectures. Where a company has publicly disclosed a detail, it is identified as a public fact. Do not treat the hypothetical diagrams as claims about a company's private stack.

Contents

1. The plan: how to learn this like a senior engineer
2. The universal cloud mental model
3. One application on a VPS, done properly
4. Infrastructure as Code: Terraform, CloudFormation, Ansible, images
5. From modular monolith to multiple applications and microservices
6. Kubernetes without mythology
7. Production delivery: CI, CD, schemas, rollback, progressive delivery
8. Observability and SRE
9. Security and software supply chain
10. Replit/E2B-style cloud: build a runtime, not just an app
11. ChatGPT-style product architecture: many execution planes
12. Multi-cloud and multi-region
13. Canary, blue/green and A/B testing
14. The nested control loops mental model
15. Senior architecture decision framework
16. Common architecture mistakes and the senior response
17. Five projects that make the concepts real
18. Tool map and official reading links
19. Final memory sheet

Part I - The plan: how to learn this like a senior engineer

What you should be able to do after this manual

A strong cloud engineer is not the person who remembers the largest number of product names. A strong engineer can take a workload and answer, in order:

1. **What is the workload?** Request/response, background jobs, streams, batch, interactive terminals, GPU inference, or a mixture?
2. **What state exists?** Transactional state, files, cache, streams, durable workflow state, ephemeral compute state, secrets, configuration.
3. **What must be synchronous?** Everything else should be a candidate for queueing or asynchronous execution.
4. **Where is the bottleneck likely to appear?** CPU, memory, connection count, write throughput, queue age, storage IOPS, network egress, GPU occupancy, vendor quota.
5. **What is the failure boundary?** Process, container, VM, node, availability zone, region, provider, or human/operator error.

6. **Who owns each desired state?** Terraform, Ansible/image builder, Kubernetes/ECS, GitOps controller, application code, or a custom scheduler.
7. **What reliability target is actually required?** SLO, RTO, RPO, durability, compliance, and blast radius.
8. **What is the simplest architecture that satisfies the above?** Complexity is a recurring operational cost.

The sequence matters. If you choose Kubernetes, Kafka, a service mesh, and multi-cloud before answering those questions, you are doing tool-driven architecture rather than systems engineering.

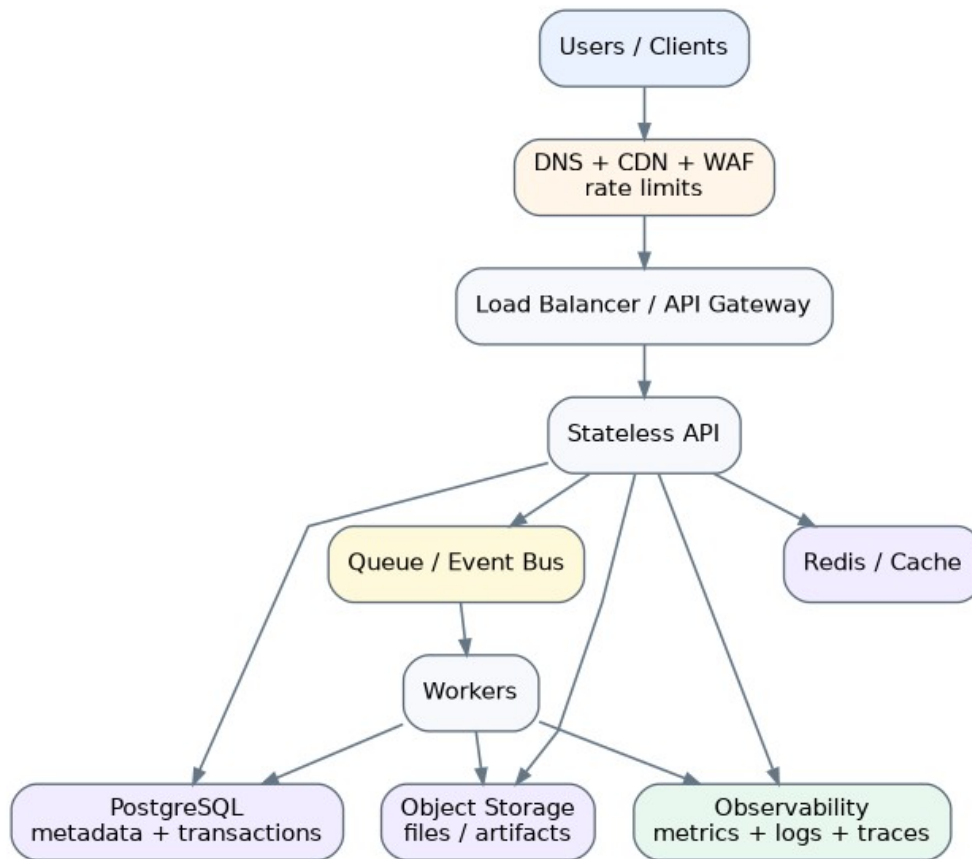
12-week progression

Weeks	Focus	Build / prove
1-2	Linux, DNS, TLS, reverse proxy, Docker, PostgreSQL, object storage	Deploy a real app on one VPS; restore it from backups
3-4	Terraform, remote state, IAM, cloud networking, managed DB	Destroy and recreate an environment reproducibly
5-6	Managed containers, queues, caches, async workers	Move long-running work off HTTP and scale workers independently
7-8	Kubernetes, HPA, probes, GitOps, progressive delivery	Run the same app on Kubernetes and safely roll versions
9-10	OpenTelemetry, SLOs, incident response, capacity, cost	Trace one request across services and run failure drills
11	Sandboxing and runtime control planes	Build a small Firecracker or VM-backed execution service
12	Capstone	Multi-region design review + A/B experiment + disaster recovery game day

Five habits that separate senior engineers

- **Measure before scaling.** Registered users are not a capacity metric. Peak RPS, concurrency, task duration, byte volume, DB QPS, queue age, GPU-seconds, and egress are.
- **Prefer managed primitives until control is strategically valuable.** Owning Kubernetes nodes, Kafka brokers, databases, or VM schedulers is justified only when the control/cost/performance benefit exceeds the operational tax.
- **Design for failure, not only success.** Every architecture diagram should have a second version showing what happens when one dependency is slow or unavailable.
- **Make change reversible.** Immutable artifacts, feature flags, canaries, backward-compatible schemas, and tested rollback paths matter more than fancy topology.
- **Reduce blast radius.** Small states, cells, queues, tenant limits, regional boundaries, and least privilege keep one failure from becoming the company's failure.

Part II - The universal cloud mental model



The universal cloud stack

Almost every product can be reduced to a small collection of primitives:

- **Edge:** DNS, CDN, TLS termination, DDoS protection, WAF, rate limiting.
- **Entry:** L4/L7 load balancer, API gateway, ingress.
- **Compute:** VM, container, serverless function, Kubernetes pod, microVM, GPU worker.
- **Transactional state:** PostgreSQL/MySQL or another database with a clear consistency model.
- **Object state:** S3/GCS/Azure Blob for files, uploads, artifacts, checkpoints, model weights.
- **Cache:** Redis/Memcached/local caches to reduce latency and backend load.
- **Messaging:** SQS/Pub/Sub/Kafka/NATS/Event Hubs to decouple producers from consumers.
- **Observability:** metrics, logs, traces, profiles, alerts, SLOs.
- **Identity/security:** IAM, workload identity, secrets, KMS, network policy, audit trails.

The specific vendor can change while the architectural role remains the same. S3, GCS and Azure Blob are implementations of object storage. ECS, Cloud Run and Azure Container Apps are managed container execution. Once you recognize the primitive, a new cloud becomes learnable rather than intimidating.

Control planes versus data planes

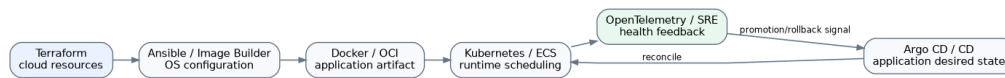
A **control plane** decides what should exist and where it should run. A **data plane** handles the actual user traffic or workload.

Examples:

- Kubernetes API/controller/scheduler = control plane; pods handling requests = data plane.
- A sandbox scheduler = control plane; Firecracker microVMs executing user code = data plane.
- Route configuration and service discovery = control plane; packets flowing through proxies = data plane.

This distinction explains why Terraform should create the durable fleet and networking, while a real-time scheduler should create user sandboxes. Terraform is not a millisecond-latency runtime scheduler.

The ownership map



Who owns which state

Memorize this separation:

Layer	Question it answers	Typical owners
Infrastructure	What cloud resources should exist?	Terraform, CloudFormation, Pulumi, CDK
Machine image/config	What should be installed/configured on a host?	cloud-init, Packer/Image Builder, Ansible
Application artifact	What exact bytes are we deploying?	Docker/OCI, package registry, binary artifact
Runtime scheduling	How many instances run, and where?	Kubernetes, ECS, Nomad, custom scheduler
Deployment reconciliation	Which application version should each environment run?	Argo CD, Flux, CI/CD platform
Reliability feedback	Is reality healthy enough to continue?	OpenTelemetry, Prometheus, SLO analysis, on-call

Senior rule: one source of truth per resource. If Terraform, a human console click, CloudFormation, and Ansible all believe they own the same load balancer, drift and surprise are inevitable.

Part III - Case 1: one application on a VPS, done properly

Suppose you are building an iLovePDF-like service. Users upload PDFs, request merge/compression/OCR/AI extraction, watch progress, and download the result.

Start smaller than your ego wants

A strong first production version can be:

Cloudflare

|

Caddy / Nginx on VPS

|

Docker Compose

| -- web/API

| -- background worker

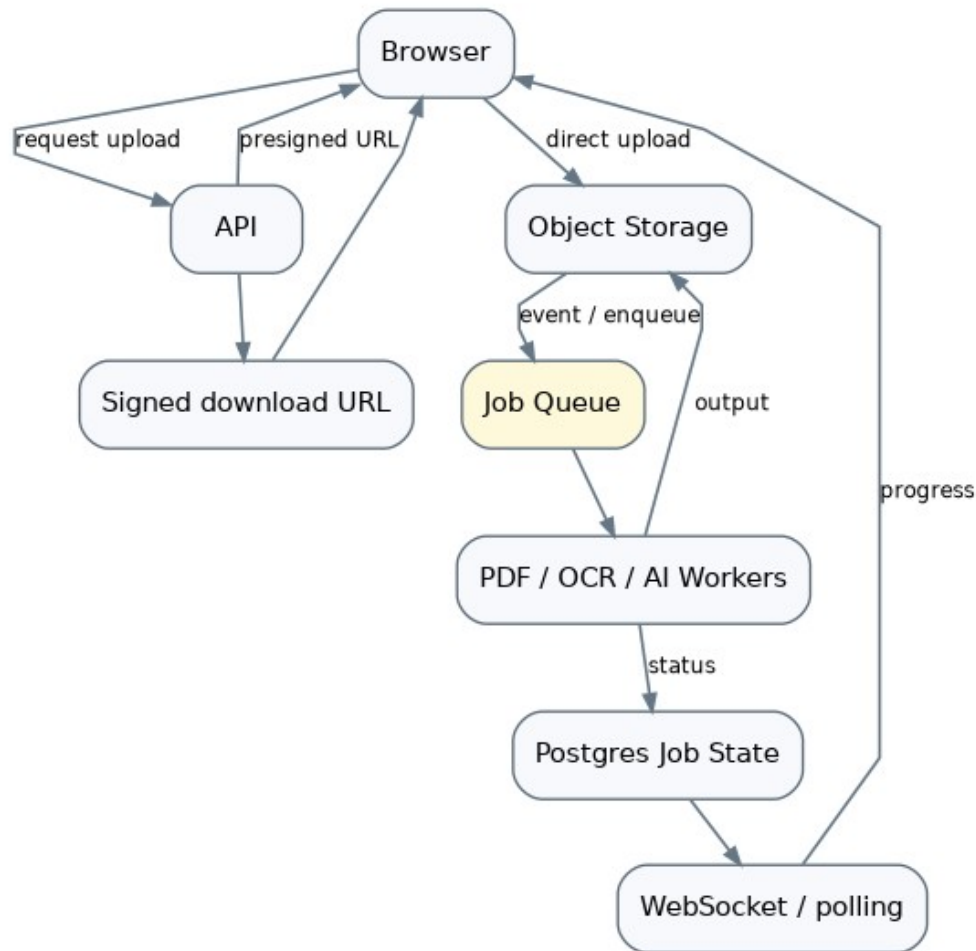
| -- Redis (or managed Redis)

|

Managed PostgreSQL Object storage (S3/R2/GCS)

You do **not** need Kubernetes simply because the word “production” appears in the requirement. At this stage Kubernetes can create more operational surface than product value.

The important design: direct upload + asynchronous processing



Asynchronous PDF processing

A bad design is:

browser -> API server -> stream 2 GB file -> process for 3 minutes -> response

The API becomes a file proxy and a long-running compute worker. Timeouts, memory pressure and retries become coupled.

A better flow is:

1. Browser asks the API for an upload slot.
2. API authenticates user, creates a job record, and returns a time-limited presigned object-storage URL.
3. Browser uploads directly to object storage.
4. Upload completion creates a job/event.

5. A worker claims the job and performs PDF/OCR/AI work.
6. Worker writes result to object storage and updates job state.
7. Browser sees status over polling/SSE/WebSocket and gets a signed download URL.

S3 presigned URLs are specifically designed to grant time-limited object access without giving the client AWS credentials. See [AWS S3 presigned URL documentation](#).

Why a queue matters

A queue is a **shock absorber**.

Imagine normal traffic is 10 conversions/second, but a campaign causes 1,000 jobs/second for three minutes. If each request directly spawns processing, every downstream dependency sees the full spike. With a queue, intake can be fast while consumption is controlled.

A production worker must assume delivery can be repeated. For an at-least-once queue, design the handler as idempotent:

```
message(job_id=J123) arrives
-> acquire/verify job state
-> if result for J123 already committed: return success
-> otherwise perform work
-> atomically mark output committed
```

For SQS, the visibility timeout keeps a received message temporarily hidden while a consumer processes it; set it with the task duration and retry behavior in mind. See [AWS SQS visibility timeout](#).

Do you need Terraform for this production VPS?

Not because it is production. Terraform earns its place when reproducibility, collaboration, drift control, security policy, or number of environments makes manual configuration risky.

Ask: if the VPS and cloud account resources vanished, can I rebuild the environment from version-controlled definitions without remembering console clicks?

For a serious small product, a reasonable Terraform scope is:

Terraform owns:

- VPC/VNet and firewall/security groups
- VM/VPS or container service
- managed PostgreSQL
- object-storage bucket
- queue
- IAM roles/policies
- DNS records where practical
- observability resources where practical

The application deployment can remain Docker Compose or a managed container service.

When Ansible is useful

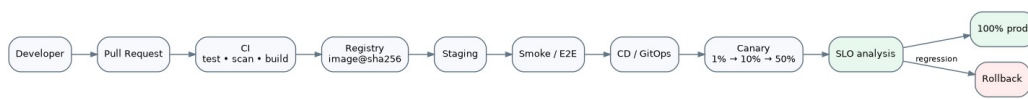
Terraform creates a machine. Ansible configures what lives inside a machine.

For a VPS or bare-metal host, Ansible can consistently apply:

- users and SSH policy
- Docker/container runtime
- kernel/sysctl settings
- firewall/host security
- Firecracker/KVM packages
- monitoring agents
- log rotation
- baseline hardening

Containers reduce the need to install application dependencies on the host. If the service runs on Fargate/Cloud Run, host configuration is largely the cloud provider's responsibility, so Ansible may disappear entirely.

A sane small-company deployment pipeline



CI/CD pipeline

A commit should move through a chain of trust:

PR

- > lint + unit tests + type checks
- > integration tests
- > dependency/SAST/container scan
- > build immutable image
- > push image@digest
- > deploy staging
- > smoke/E2E test
- > production promotion
- > health/SLO verification
- > rollback if regression

Use immutable image digests or commit SHAs. `myapp:latest` is convenient for humans but weak as production evidence. You want to know exactly which artifact is running.

GitHub Actions supports OIDC federation so workflows can obtain short-lived cloud credentials instead of storing long-lived cloud access keys. See [GitHub OIDC for cloud providers](#).

The first production checklist

Before worrying about Kubernetes, make this boring architecture genuinely reliable:

- automated off-host backups and tested restore
- database migration strategy
- TLS and secure headers
- rate limiting and upload size limits
- object lifecycle/expiry for temporary PDFs
- idempotent jobs and a dead-letter path

- disk-space and memory alerts
- application health endpoint
- structured logs with request/job IDs
- deploy rollback to the previous artifact
- secret storage outside Git
- least-privilege IAM
- documented incident/runbook steps

If you cannot operate this well, adding orchestration will not save you.

Part IV - Infrastructure as Code: Terraform, CloudFormation, Ansible, images

Terraform's real mental model

Terraform is a declarative state reconciler for infrastructure APIs.

configuration = desired state

state = identity/bindings + metadata Terraform remembers

provider = implementation that talks to a remote API

plan = proposed delta between desired and observed state

apply = execution of that delta

Terraform builds a dependency graph so independent resources can be processed concurrently while dependent resources wait. This is why a subnet referenced by a Kubernetes cluster naturally becomes a dependency. See [Terraform dependency graph](#).

Terraform state maps configuration resources to real remote objects. HashiCorp recommends remote state for team use and warns against treating the state file casually because it can contain sensitive values. See [Terraform state](#) and [remote state](#).

A production repository shape

infra/

modules/

network/

postgres/

object-storage/

container-platform/

observability/

live/

dev/

staging/

prod/

Modules are useful when they encode a repeated architectural unit, not merely to split every five lines into another abstraction. HashiCorp explicitly recommends moderation with modules. See [Terraform modules](#) and [creating modules](#).

Separate state by blast radius and ownership

Do not let one state file grow into “the entire company”. Reasonable independent states might be:

prod-network
 prod-shared-data
 prod-app-platform
 prod-observability

Benefits:

- smaller plan/apply scope
- fewer unrelated locks
- clearer permissions
- smaller accidental-destruction blast radius
- teams can own different lifecycle domains

The tradeoff is cross-state coordination. Avoid making every tiny output a web of remote-state dependencies; stable interfaces such as DNS names, parameter stores, service discovery or explicit pipeline outputs can be cleaner.

Plan/apply discipline

Production Terraform should normally look like:

pull request
 -> terraform fmt/validate
 -> static policy/security checks
 -> terraform plan
 -> human/policy review
 -> approved apply with short-lived identity
 -> change record/audit trail

A plan is not decorative output. Read destroys/replacements, IAM changes, network changes, and database modifications carefully.

Drift and imports

If someone creates a resource manually, decide whether it is:

- intentionally unmanaged, or
- a resource Terraform should import and own.

Do not let “we sometimes click things in the console” become an undocumented secondary deployment system.

Terraform versus CloudFormation/CDK versus Pulumi

Choice	Strong fit	Main tradeoff
Terraform/OpenTofu	Multi-provider, broad ecosystem, common platform skill	State management and provider lifecycle; HCL abstraction discipline
CloudFormation	Deep AWS-native integration, AWS-only organization	AWS-specific; verbose raw templates
AWS CDK	Teams wanting programming-language abstractions over CloudFormation	Generated CFN layer and framework abstractions must be understood

Choice	Strong fit	Main tradeoff
Pulumi	IaC with general-purpose languages and multi-cloud	Language/runtime complexity; smaller ecosystem than Terraform
Ansible	Host/OS/application configuration on existing machines	Not the ideal owner of cloud resource graphs or high-frequency runtime scheduling

No company receives bonus points for using all of them. Pick clear ownership boundaries.

Immutable machine images

For large VM fleets, repeatedly booting a base OS and running 20 minutes of configuration can create slow, divergent hosts. A stronger pattern is:

CI builds golden AMI/image

-> hardening + packages + host agent preinstalled

-> scan/test image

-> version image

-> Terraform/ASG launches that immutable version

Ansible can still be used during image creation or for small runtime configuration, but the fleet becomes more deterministic.

Terraform is not the answer to per-user runtime objects

This is a crucial boundary:

GOOD:

Terraform -> 200 sandbox host machines

runtime scheduler -> 80,000 user microVM sessions

BAD:

HTTP POST /sandbox

-> terraform apply

-> create a VM resource per user request

Terraform is optimized for infrastructure lifecycle, planning, drift and dependencies, not thousands of latency-sensitive ephemeral actions per second.

Part V - From modular monolith to multiple applications and microservices

Features do not equal services

Suppose your product has authentication, file storage, processing, AI insights, realtime progress, billing and notifications. That does not imply seven microservices.

A strong early architecture might be:

modular API / monolith

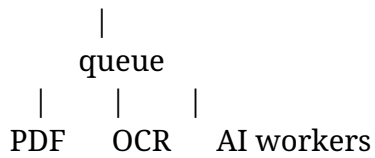
| -- auth module

| -- documents module

| -- billing module

| -- AI orchestration module

PostgreSQL + object storage + Redis

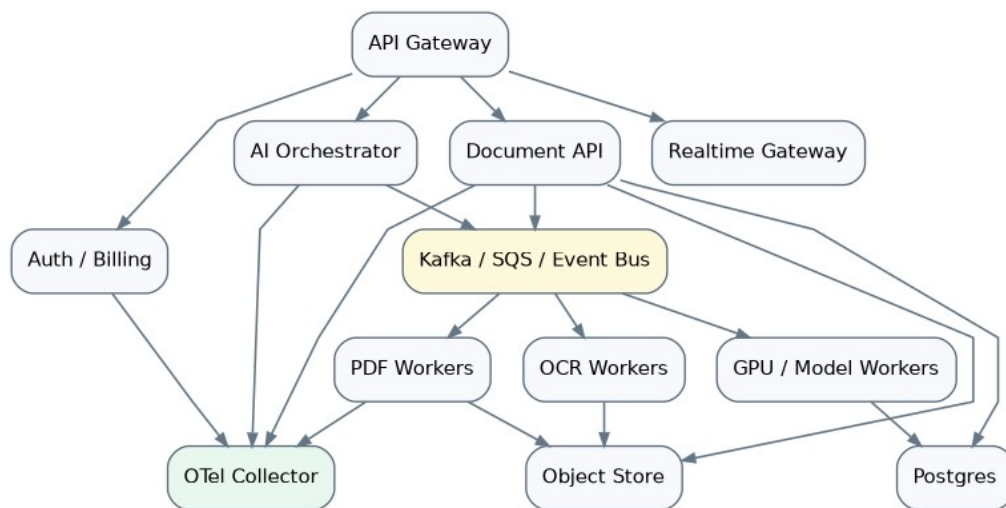


Extract a service when a boundary has a real reason:

- independent scaling profile
- independent failure isolation
- materially different runtime (GPU, Firecracker, streaming)
- ownership by a distinct team
- different security/compliance boundary
- independent release cadence

Microservices without those reasons often buy distributed transactions, network failures, versioned contracts and observability complexity before you need them.

A polyglot platform example



Event-driven multi-service architecture

A platform might use:

- TypeScript/Go API for product endpoints
- Python for AI orchestration and model tooling
- Go/Rust for high-throughput gateways or low-level host agents
- Rust/C for sandbox/runtime components

But choose a language because of runtime/library/ownership needs, not because a diagram looks more sophisticated.

Scaling should be workload-based

The following numbers are illustrative. The point is the reasoning method.

About 100 active users

Assume peak 5-20 RPS, tens of concurrent jobs, <1 TB object storage, modest DB load.

Use:

- 1-2 application instances
- managed PostgreSQL
- object storage
- queue + 1-5 workers
- Redis only if measurements justify it
- CDN/WAF
- simple CI/CD

Kubernetes is optional and usually unnecessary.

About 1,000 active/concurrent users

Assume 100-500 RPS, hundreds of concurrent background jobs, WebSockets/SSE, more strict uptime.

Consider:

- managed container platform (ECS/Fargate, Cloud Run, Azure Container Apps) or a small Kubernetes cluster if team/workload complexity already warrants it
- multiple API replicas
- connection pooling
- Redis for hot cache/rate/session state
- queue depth-based worker scaling
- read replicas only after DB analysis shows read pressure
- load tests and SLOs
- separate worker pools for CPU/GPU/long jobs

AWS describes Fargate as serverless container compute where the underlying infrastructure is managed for you; ECS also provides EC2/managed capacity choices when more control is required. See [Amazon ECS/Fargate](#).

One million registered users is not an architecture

One million registered users could mean 2,000 concurrent users and be easy. Or 50,000 users could each run a browser+code sandbox and be expensive.

For a large platform, quantify:

peak API RPS

concurrent sessions

concurrent WebSockets

upload bytes/sec

jobs/sec and p95 job duration

CPU-seconds/job

GPU-seconds/job
 DB reads/writes/sec
 cache hit ratio
 queue oldest-message age
 object storage growth/day
 regional distribution
 egress GB/day

Then scale the saturated component.

Database scaling order

Do not jump directly to sharding. A common order is:

1. Fix missing indexes and bad queries.
2. Bound N+1 patterns and huge scans.
3. Use connection pooling.
4. Cache truly hot/read-heavy data.
5. Increase instance resources if economical.
6. Add read replicas for read-heavy workloads.
7. Move suitable workloads to specialized stores.
8. Partition/shard only when the write/storage/operational requirement truly demands it.

A useful public example is OpenAI's January 2026 engineering write-up: OpenAI reported using a single primary Azure PostgreSQL Flexible Server plus nearly 50 read replicas to serve read-heavy core product workloads at very large scale, while moving some write-heavy shardable workloads to other systems. See [OpenAI: Scaling PostgreSQL to power 800 million ChatGPT users](#).

The lesson is not “copy OpenAI.” It is: **optimize the workload you actually have before replacing the database architecture you understand.**

Backpressure and load shedding

At scale, “autoscale everything” is insufficient. Autoscaling has lag. A spike can kill dependencies before new capacity arrives.

Use:

- bounded queues
- concurrency limits per worker
- per-tenant quotas
- request deadlines
- retry budgets with exponential backoff + jitter
- circuit breakers for failing dependencies
- admission control
- graceful degradation
- load shedding of noncritical features

Example: if the AI insights system is saturated, document upload might continue while “deep analysis” is queued or temporarily unavailable. Protect the core transaction rather than letting every feature fail together.

Part VI - Kubernetes without mythology

Kubernetes is a desired-state runtime control system for containers. A Deployment describes how stateless replicas should look; controllers continuously reconcile reality toward that desired state. See [Kubernetes Deployments](#).

What Kubernetes gives you

- scheduling across nodes
- restart/rescheduling on failure
- service discovery and load balancing
- declarative rollouts
- resource requests/limits
- horizontal autoscaling
- storage integration
- policy/identity/network ecosystem
- a common API on which platform teams can build

What Kubernetes does not magically give you

- good application architecture
- correct database schemas
- instant autoscaling
- safe retries
- observability
- sensible costs
- multi-region failover
- correct security defaults
- freedom from operations

Probes: readiness is not liveness

Kubernetes uses readiness to decide whether a pod should receive Service traffic, liveness to decide when a stuck container should be restarted, and startup probes to protect slow-starting applications. Poorly designed liveness probes can cause cascading restarts under load. See [Kubernetes probes](#).

A useful pattern:

- `/ready`: can this instance serve new requests now?
- `/live`: is the process irrecoverably unhealthy?
- startup probe: has expensive initialization completed?

Do not make liveness depend on every downstream service. If your database is briefly slow and every API pod kills itself, you made the outage larger.

Two autoscaling loops

traffic/job pressure rises

- > HPA/KEDA asks for more pods
- > scheduler tries to place pods
- > if cluster lacks capacity, node autoscaler creates nodes
- > new pods start and become ready

Kubernetes HPA scales workloads horizontally according to observed metrics. See [Horizontal Pod Autoscaling](#).

Pod scaling and node scaling are different control loops. A senior engineer thinks about:

- metric delay
- pod startup time
- node provisioning time
- warm capacity/headroom
- scale-down safety
- queue-age target
- startup storms against databases

StatefulSet is not “run every database in Kubernetes”

StatefulSet provides stable identity/order/persistent-volume behavior for stateful pods. It is useful when the workload needs those properties. See [Kubernetes StatefulSets](#).

For a small/medium team, managed PostgreSQL often remains operationally superior to self-running a critical database in Kubernetes.

GitOps

Argo CD treats Git as the source of truth for application desired state and continuously compares it with the live Kubernetes state. See [Argo CD](#).

The clean model is:

application repo

- > CI produces image digest
- > deployment-config repo references digest
- > Argo CD notices desired-state change
- > cluster reconciles

This separates **CI** (can we produce a trusted artifact?) from **CD** (should this environment run this artifact?).

Part VII - Production delivery: CI, CD, schemas, rollback, progressive delivery

Build once, promote the same artifact

Do not rebuild a different image for production after staging passed. Build an immutable artifact once and promote that exact digest through environments.

A mature pipeline can include:

- lint, formatting, unit tests

- type checking
- integration/contract tests
- secret scanning
- SAST and dependency scanning
- Docker multi-stage build
- image vulnerability scan
- SBOM generation
- artifact signing/attestation
- staging deploy
- migration compatibility checks
- smoke/E2E/load tests
- progressive production rollout
- automatic/assisted rollback

Docker recommends multi-stage builds to separate build dependencies from the final runtime image, reducing size and attack surface. See [Docker build best practices](#).

Database changes must survive mixed versions

Canary deployments mean old and new application versions may run at the same time. Therefore:

Bad migration: rename/drop a column and immediately deploy code that assumes the new schema.

Expand-contract pattern:

1. Add new schema in a backward-compatible way.
2. Deploy code that can use old + new paths.
3. Backfill data.
4. Switch reads/writes.
5. Verify.
6. Remove old column/contract in a later deployment.

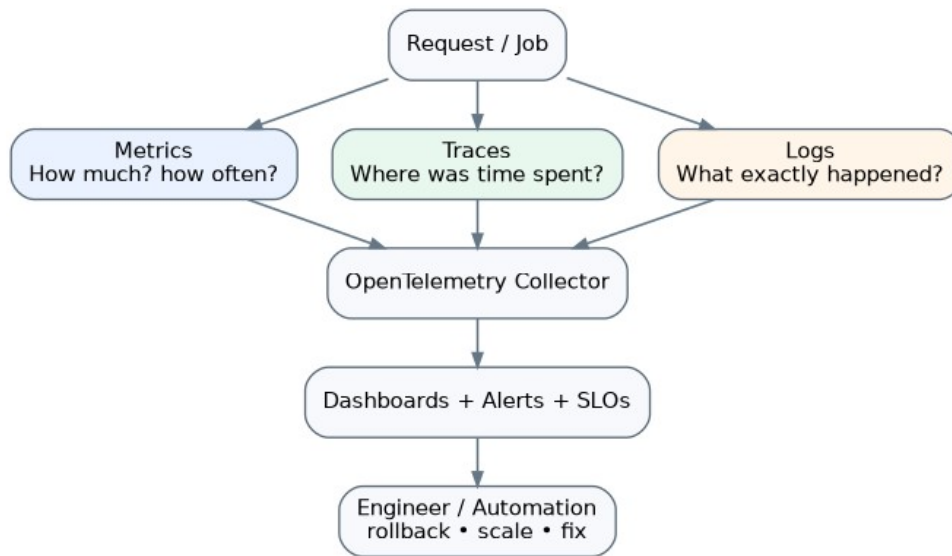
This is how you make rollback possible without database panic.

Blue/green versus canary

- **Blue/green:** prepare a full new stack/version, verify it, then switch traffic. Simple mental model, quick rollback, but doubles capacity during transition and exposes the new version to traffic more abruptly.
- **Canary:** send a small percentage to the new version, evaluate metrics, then increase. Lower blast radius but requires trustworthy metrics, compatible versions and often better traffic routing.

Argo Rollouts supports both strategies and can run metric-driven analysis before/after promotion. See [Argo Rollouts canary](#) and [blue/green](#).

Part VIII - Observability and SRE: know what failed and why



Metrics, logs, traces

OpenTelemetry is a vendor-neutral framework for generating, collecting and exporting telemetry such as traces, metrics and logs. The Collector provides a common receive/process/export layer. See [OpenTelemetry documentation](#) and [OTel Collector](#).

The three basic questions

Metrics: Is the system behaving abnormally? How often and how much?

Examples: request rate, p95/p99 latency, error rate, CPU, DB connections, queue age, job success.

Traces: Where did a particular request/job spend its time?

A trace might show:

```

POST /analyze 4.2s
  auth      18ms
  postgres  44ms
  object read 120ms
  AI provider 3.8s <-- bottleneck
  persistence 90ms
  
```

Logs: What happened at the event level?

Logs need stable correlation fields: request ID, job ID, tenant ID, trace ID, version, region. Avoid logging secrets or raw sensitive user content by default.

RED and USE

For request-driven services, start with **RED**:

- Rate
- Errors

- Duration

For resources, use **USE**:

- Utilization
- Saturation
- Errors

This prevents dashboards with 100 graphs but no operational answer.

Product SLOs, not just host uptime

For a PDF service:

SLI: percentage of valid conversion jobs completed successfully within 30s

SLO: 99.9% over 30 days

For a sandbox platform:

SLI: successful sandbox boots under 2s

SLO: 99.5% over 28 days

Google SRE defines an error budget as 1 - SLO; it is a mechanism for balancing reliability and change velocity. See [Google SRE error budget policy](#).

Alert on symptoms and imminent exhaustion

Good pages wake a human for conditions requiring human action:

- user-visible error SLO burn
- sustained p99 latency breach
- oldest queue message exceeds target
- DB connection pool near exhaustion
- replication lag threatens correctness/read freshness
- regional capacity headroom below safety margin

A warning that one pod restarted once is usually context, not a 3 a.m. page.

Incident loop

detect -> mitigate -> communicate -> recover -> learn -> prevent recurrence

During an incident, prioritize reducing user impact over proving the root cause. Afterwards, write a blameless technical postmortem that identifies contributing conditions and changes the system so the same class of failure is harder to repeat.

Part IX - Security and software supply chain

The minimum serious baseline

- least-privilege IAM roles per service
- short-lived workload identity instead of static cloud keys
- secrets manager/Vault rather than secrets in images or Git
- encryption in transit and at rest

- private networking for databases/internal services where appropriate
- WAF/rate limits and abuse controls at the edge
- dependency and image scanning
- SBOM and artifact provenance for important systems
- protected deployment environments and approvals
- audit logs for production changes
- tested credential rotation

The AWS Well-Architected Framework is useful because it forces explicit tradeoff review across operational excellence, security, reliability, performance efficiency, cost optimization and sustainability. See [AWS Well-Architected](#).

Rate limiting is architecture, not middleware trivia

Use multiple levels:

edge IP/device limit

-> authenticated user/tenant quota

-> expensive endpoint limit

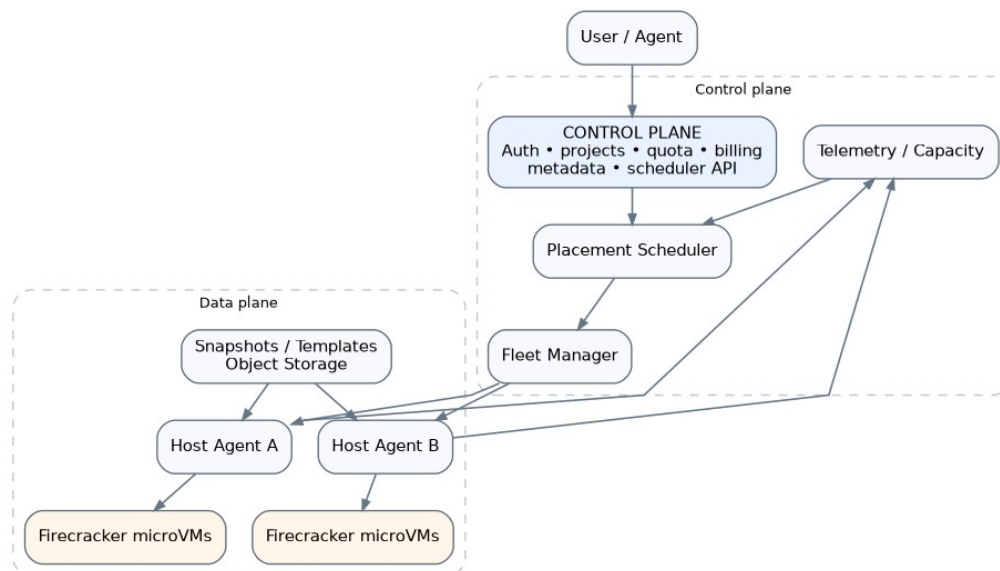
-> downstream concurrency limit

-> provider quota budget

For AI/sandbox products, one abusive or accidentally looping tenant must not consume the entire GPU/sandbox pool.

Part X - Replit/E2B-style cloud: build a runtime, not just an app

This is where ordinary application engineering turns into infrastructure-product engineering.



Sandbox control plane and data plane

The two planes

Control plane:

- users/organizations/projects
- authentication and API
- quotas and billing
- sandbox metadata
- templates/images/snapshots
- placement scheduler
- fleet inventory and health
- region selection
- lifecycle state

Data plane:

- physical/VM hosts with KVM
- Firecracker processes/microVMs
- filesystems
- terminal/PTY
- port forwarding/proxy
- network policy/egress
- persistent volumes if offered

Sandbox creation flow

A request like:

```
{ "template": "python", "cpu": 2, "memory_mb": 2048, "region": "eu" }
```

should become:

API

- > authenticate + quota
- > create lifecycle record
- > scheduler scores eligible hosts
- > reserve capacity
- > host agent receives launch command
- > fetch/locate snapshot/template
- > create network/filesystem resources
- > launch Firecracker process via jailer
- > wait for guest readiness
- > expose terminal/port route
- > mark session RUNNING

Terraform is nowhere in that synchronous request path.

What Terraform does here

Terraform can create:

- VPCs/subnets/routing
- regional control-plane clusters
- database/cache/object storage
- sandbox host autoscaling groups or base instances
- IAM/KMS
- global/region DNS
- telemetry backend resources

For E2B's current public material, E2B describes each sandbox as a microVM and offers cloud/BYOC deployment options; its security page says managed sandboxes run on Google Cloud and BYOC can run inside customer AWS/GCP/Azure accounts. See [E2B](#) and [E2B security](#).

Why Firecracker

Firecracker uses KVM to create lightweight microVMs intended for multi-tenant workloads. Its design uses multiple isolation layers; Firecracker's documentation recommends production use through the [jailer](#), and default seccomp filters restrict host syscalls. See [Firecracker design](#), [jailer](#), and [seccomp](#).

A simplified host:

Linux host

KVM

```
|-- Firecracker process -> microVM A -> user task A
|-- Firecracker process -> microVM B -> user task B
|-- Firecracker process -> microVM C -> user task C
```

Each sandbox still requires defense in depth:

- resource cgroups/quotas
- namespace/jailer isolation
- network egress policy
- metadata/credential protection
- filesystem lifecycle
- session timeout
- host patching
- abuse detection
- no host/cloud credentials exposed to guest

Snapshots and startup speed

A runtime does not want every session to boot an OS and reinstall packages from scratch. Templates and snapshots can reduce startup latency dramatically.

Firecracker snapshotting serializes microVM state and guest memory for later restoration; block devices are managed separately. See [Firecracker snapshot support](#).

A practical hierarchy:

base kernel/rootfs

-> language template

-> project/tool template

- > warm regional cache
- > session-specific writable overlay

Scheduling is the product

A naive scheduler finds any host with free RAM. A real scheduler considers:

hard constraints:

- enough CPU/RAM/GPU
- compatible host architecture
- allowed region/compliance zone
- required template/snapshot capability

soft score:

- + snapshot already cached
- + low current CPU pressure
- + enough memory headroom
- + same region as user/data
- host failure risk
- expensive fragmentation

You also need reservation semantics. Without reserving capacity atomically, 100 schedulers can all observe the same “free” 4 GB and oversubscribe the host.

Scale stages

100 users / dozens of concurrent sandboxes

- 1 control-plane API
- PostgreSQL + Redis
- object storage for templates
- 2-4 KVM hosts
- host agent
- simple scheduler
- WebSocket/PTY gateway
- Terraform + image builder/Ansible for host fleet

1,000+ concurrent sandboxes

Add:

- regional scheduler/fleet manager
- warm template/snapshot caches
- capacity reservations
- host draining and rolling upgrades
- per-tenant quotas
- autoscaling host pools
- more robust proxy/port routing
- queue for launches if capacity temporarily unavailable
- dedicated capacity/health telemetry

Very large scale

Move toward **cells**:

global control layer

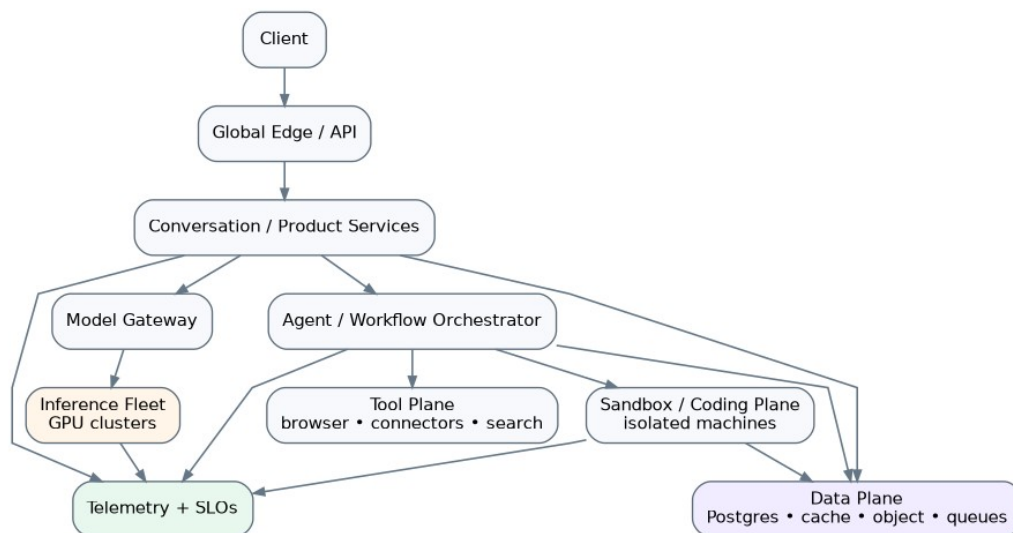
- |-- US cell: scheduler + cache + host fleet
- |-- EU cell: scheduler + cache + host fleet
- |-- APAC cell: scheduler + cache + host fleet

A cell is a deliberate failure domain. One bad rollout, cache incident or capacity bug should not break every region and tenant simultaneously.

Part XI - ChatGPT-style product architecture: many execution planes

This section separates **public facts** from a **reference architecture**.

Public fact: OpenAI has described very large core product workloads backed by PostgreSQL with one primary and many read replicas, and has published infrastructure roles related to large GPU fleets. The exact private end-to-end ChatGPT architecture is not public. The following diagram is therefore a teaching architecture for a product with chat, model inference, tools, cloud coding and long-running work.



Chat/agent/model/runtime planes

Plane 1 - Product/API plane

Responsibilities:

- auth/account/entitlements
- conversation metadata
- rate limits/quotas
- request routing
- streaming responses
- file/artifact references

This plane should be mostly stateless at compute level so instances can be horizontally replicated.

Plane 2 - Model gateway and inference plane

A model gateway can handle:

- model selection
- safety/policy routing
- batching
- provider/fleet selection
- context/token accounting
- retries/deadlines
- streaming
- admission control

The inference fleet has different scaling economics from the API: GPU memory, batching efficiency, model weights, prefill/decode behavior and expensive cold capacity dominate.

Plane 3 - Agent/workflow plane

A simple chat request may complete in seconds. An agent task can run for minutes or hours and include many tool calls. That means you need durable workflow semantics:

start task

- > plan/tool call
- > checkpoint state
- > wait for browser/sandbox/API
- > checkpoint
- > retry transient failure
- > ask for user approval if needed
- > continue from durable state
- > produce artifact/result

A workflow engine such as Temporal/Step Functions/Durable Functions is a useful architectural reference because durable state and retries should not depend on one worker process remaining alive.

Plane 4 - Coding/sandbox plane

A Codex-like task has a fundamentally different execution model from pure inference:

agent orchestration

- > acquire isolated runtime
- > clone/mount repository
- > execute shell/tools/tests
- > edit files
- > maintain checkpoints/logs
- > enforce network/secret policy
- > terminate/recycle sandbox

This resembles the Replit/E2B control/data-plane model.

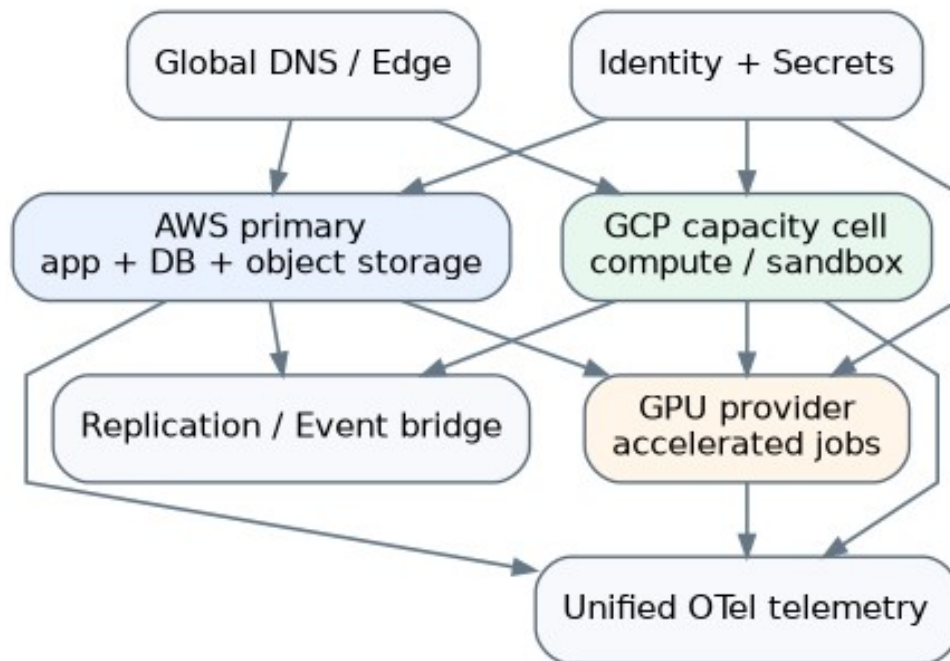
Plane 5 - Browser/computer-use plane

Browser/computer tasks add:

- virtual desktop/browser instances
- session persistence
- credential/cookie boundaries
- screenshots/DOM/tool observations
- egress and anti-abuse controls
- approval gates around consequential actions

Do not force every execution type into the same worker pool. Different workloads need different isolation, timeouts, autoscaling and cost controls.

Part XII - Multi-cloud and multi-region: choose deliberately



Selective multi-provider architecture

Terraform can provision resources from multiple providers, but a shared IaC syntax does not erase provider differences.

AWS IAM != Azure Entra/IAM != GCP IAM

S3 semantics != every Blob/GCS behavior

EKS != AKS != GKE

cross-cloud network != same-VPC network

egress is not free

quotas, regions and failure modes differ

Usually choose this progression

single cloud

-> multi-AZ

-> well-tested backup/restore

-> multi-region if the business needs it

-> selective multi-provider if it solves a concrete risk/capacity/commercial problem

Practical multi-provider patterns

Pattern A: edge + primary cloud. Cloudflare at edge, AWS as application/data cloud. Technically multi-provider, operationally sane.

Pattern B: specialized capacity. Main product on AWS/GCP; GPU jobs burst to a specialized GPU provider.

Pattern C: BYOC/data-residency. Your control plane operates customer-local data planes in their cloud account.

Pattern D: active-passive disaster recovery. Primary provider/region serves traffic; secondary environment has replicated critical state and a tested failover path.

Pattern E: active-active multi-cloud. Both providers serve live traffic. This is expensive in networking, data consistency, identity, operations and testing. Use only with a clear business justification.

RTO and RPO

Before saying “multi-region,” define:

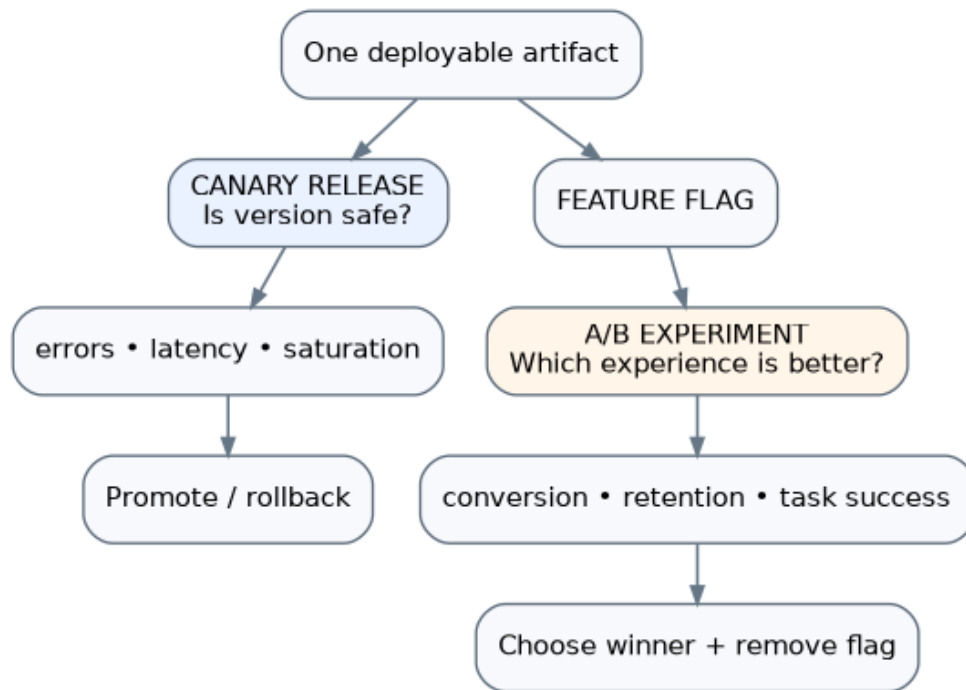
- **RTO (Recovery Time Objective):** how long can the service be unavailable?
- **RPO (Recovery Point Objective):** how much recently written data can the business afford to lose?

A nightly backup gives roughly a one-day worst-case RPO; that is very different from synchronous/near-synchronous replication.

Data gravity

Compute is comparatively easy to move. Large state is not. A multi-cloud strategy that requires moving terabytes across providers per day can create high egress cost and large latency/consistency complexity. Place compute near durable data unless there is a strong reason not to.

Part XIII - Canary, blue/green and A/B testing: do not confuse them



Reliability rollout versus product experiment

This distinction is essential.

Canary release asks: “Is the new version safe?”

You deploy version B to a small production slice and inspect **operational guardrails**:

- 5xx rate
- p95/p99 latency
- CPU/memory saturation
- DB pressure
- job failure rate
- crash rate

If B is unhealthy, roll back. You are primarily managing release risk.

Example:

v1 95% traffic

v2 5% traffic

wait 10 min

if `error_rate(v2) <= threshold` and p99 acceptable:

25%

50%

100%

else rollback

Argo Rollouts provides weighted canary steps, pauses and analysis runs. It also provides an Experiment CRD that can run baseline and canary ReplicaSets for analysis. See [Argo Rollouts experiments](#).

A/B experiment asks: “Which experience causes the better outcome?”

Both A and B can be technically healthy. You measure a **causal/product outcome**:

- signup conversion
- retention
- checkout completion
- task success
- user time-to-complete
- revenue per eligible user
- AI answer acceptance rate

Feature flags are a common mechanism. OpenFeature defines a vendor-neutral feature flag API; feature flags can alter behavior without a deployment and support use cases such as controlled rollout and A/B testing. See [OpenFeature](#).

Correct experiment architecture

request(user_id)

- > eligibility check
- > stable randomization / sticky assignment
- > variation A or B
- > record exposure event
- > user generates outcome event
- > analytics joins exposure -> outcome
- > compare primary metric + guardrails

Why sticky assignment matters

If the same user randomly sees A on Monday and B on Tuesday, treatments contaminate each other. Hash a stable randomization key such as user/account ID into buckets and keep assignment stable for the experiment.

Exposure matters more than assignment

Do not count a user as “treated” merely because they were eligible. Record exposure when the code actually evaluated/served the experimental experience. Otherwise metrics can be diluted by users who never saw the feature.

Pick metrics before looking at results

Define:

- hypothesis
- randomization unit
- primary metric
- guardrail metrics
- experiment population
- expected runtime/sample requirement
- stopping rule

If you keep changing the metric after inspecting data, you are optimizing the story rather than testing a hypothesis.

Run A/A tests

An A/A test gives both groups identical behavior. It can reveal broken bucketing, event loss, sample ratio mismatch, or biased data pipelines before you trust A/B results. LaunchDarkly's experimentation documentation explicitly describes A/A tests as a way to validate experiment setup. See [LaunchDarkly experiment design](#).

A/B testing infrastructure changes

You can also compare backend strategies if user behavior is not the only metric:

50% of eligible jobs -> worker implementation A
 50% -> worker implementation B

measure:

- success rate
- p95 runtime
- cost/job
- CPU/GPU seconds
- output-quality score

But be careful: if treatment changes shared state or a cache, the groups may interfere with each other.

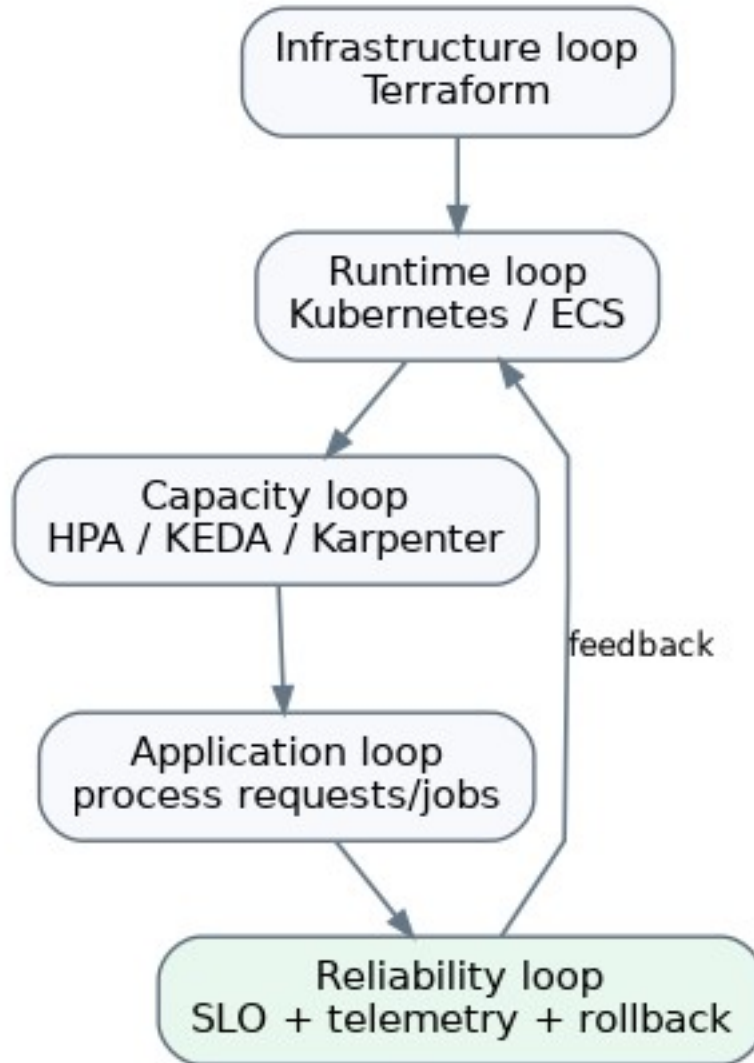
Deployment + experiment together

A robust feature launch can use both:

1. Deploy code containing both control and treatment behind a flag.
2. Canary the new code version for operational safety.
3. Promote code to 100% when healthy.
4. Run A/B experiment by flag assignment.
5. Choose winner.
6. Roll winning behavior to 100%.
7. Delete temporary flag and dead code.

The release system protects reliability. The experiment system learns product impact.

Part XIV - The nested control loops mental model



Nested production control loops

A modern production platform is a set of nested feedback loops:

1. **Terraform loop:** durable cloud infrastructure should match declared infrastructure.
2. **Runtime loop:** the orchestrator should maintain requested workloads.
3. **Capacity loop:** replicas/nodes should adapt to demand while preserving headroom.
4. **Application loop:** services process user requests and queued work safely.
5. **Reliability loop:** telemetry and SLOs decide whether to promote, scale, degrade or roll back.

Once you see systems this way, tools stop looking like unrelated products. Each one is a controller acting on a particular state with a particular timescale.

Timescale matters

Control loop	Typical reaction time
Request timeout/circuit breaker	milliseconds-seconds

Control loop	Typical reaction time
Worker concurrency / queue consumption	seconds
Pod autoscaling	tens of seconds-minutes
Node/fleet scaling	minutes
Terraform infrastructure changes	minutes-hours, controlled changes
Multi-region capacity planning	days-weeks

A slow control loop cannot protect you from a fast overload by itself. That is why queueing, concurrency limits and load shedding matter even if autoscaling exists.

Part XV - Senior architecture decision framework

Before you select technologies, write an architecture decision record answering these.

Workload

- synchronous requests or asynchronous jobs?
- request rate and burst factor?
- concurrency?
- CPU/memory/GPU profile?
- data size and locality?
- task duration distribution?

State

- which data needs ACID transactions?
- which data can be eventually consistent?
- which data is immutable/blob-like?
- which state is ephemeral?
- retention requirements?

Reliability

- user-facing SLO?
- RTO/RPO?
- allowed degradation modes?
- highest acceptable blast radius?
- dependency timeout/retry policy?

Security

- trust boundaries?
- tenant isolation requirement?
- credential lifecycle?
- audit/compliance/data residency?
- untrusted code execution?

Operations

- team size and skill?
- who is on call?
- can the system be recreated?
- can a deploy be rolled back?

- can the DB schema survive mixed versions?
- is failure observable end-to-end?

Cost

- cost per request/job/sandbox-hour?
- idle capacity?
- egress?
- logs/metrics cardinality?
- database I/O/storage?
- GPU utilization?

The final selection question

What is the **least complex architecture** that meets the measured workload, security boundary, reliability target and team-operability constraint for the next meaningful growth horizon?

Not the next ten years. Enough runway that you are not re-platforming every month, without operating a hyperscaler architecture before you have hyperscaler problems.

Part XVI - Common architecture mistakes and the senior response

Temptation	Senior response
"We have many features, so microservices."	Features are modules first. Extract services for scaling/ownership/failure boundaries.
"Production means Kubernetes."	Production means reliable operations. Use the smallest platform that meets requirements.
"Terraform can provision anything, so use it for sandboxes."	Terraform owns durable infrastructure; runtime schedulers own high-frequency ephemeral sessions.
"Autoscaling will handle spikes."	Autoscaling has lag. Add queues, headroom, concurrency limits and load shedding.
"Multi-cloud prevents outages."	It also multiplies identity, data, networking, release and testing complexity. Define the failure you need to survive.
"Read replicas fix the database."	First inspect queries, indexes, connection behavior and workload shape.
"Retries improve reliability."	Retries without deadlines, idempotency, backoff and budgets can amplify outages.
"Liveness probe checks every dependency."	That can cause restart storms. Readiness and liveness answer different questions.
"A/B means send 50/50 traffic."	You need stable randomization, exposure events, predeclared metrics and statistical discipline.
"Observability means logs."	Metrics locate symptoms, traces locate paths, logs explain events. SLOs decide priority.

Part XVII - Five projects that will make the concepts real

Project 1 - Reproducible production SaaS on one VM

Goal: prove you can operate a simple system excellently before adding orchestration.

Product: URL/file utility service with login, file upload and asynchronous conversion.

Architecture:

Cloudflare -> Caddy -> Docker Compose API/worker
 | -> managed Postgres
 | -> S3-compatible object storage
 | -> SQS/Redis queue

Tools: Terraform, Docker, GitHub Actions, managed PostgreSQL, S3/R2, Caddy, OTel/Sentry.

Required deliverables:

- infra/ Terraform with remote state
- one-command/dev-environment bootstrap
- CI with tests + image build + OIDC deploy
- presigned direct upload
- idempotent worker and DLQ strategy
- backup + documented restore drill
- dashboard with RED metrics
- rollback to prior image

Failure drills: kill API container; kill worker; fill local disk; make object storage slow; break a deployment; restore DB into a new environment.

You pass when: you can rebuild the service from code and explain every stateful component.

Project 2 - Event-driven document/AI platform

Goal: learn independent scaling, queues, contracts and managed compute.

Product: upload PDF -> OCR -> extraction -> AI insights -> realtime progress.

Architecture: API + worker pools + queue/event bus + object storage + PostgreSQL + Redis.

Recommended first runtime: ECS/Fargate or Cloud Run rather than Kubernetes, so you learn distributed application patterns before cluster operations.

Required experiments:

- scale PDF workers based on queue depth
- introduce poison jobs and DLQ
- enforce per-tenant concurrency quotas
- load test API separately from workers
- measure cost/job and p95 job completion
- add read caching only after proving DB pressure

Stretch: split AI orchestration into a separate service and add contract tests.

Project 3 - Kubernetes + GitOps + progressive delivery lab

Goal: understand Kubernetes as a controller system rather than YAML memorization.

Tools: kind/k3d locally, then EKS/GKE/AKS; Helm/Kustomize; Argo CD; Argo Rollouts; Prometheus/Grafana; OTel; KEDA/Karpenter where appropriate.

Build: deploy Project 2 on Kubernetes.

Required work:

- Deployments with requests/limits
- correct readiness/liveness/startup probes
- HPA on API
- queue-based worker autoscaling
- PodDisruptionBudget where justified
- Argo CD GitOps repo
- canary 5% -> 25% -> 50% -> 100%
- automatic rollback on error/latency threshold
- backward-compatible database migration

Failure drills: delete pods; drain a node; inject 2s DB latency; make readiness fail; create a bad liveness probe and observe cascading behavior; exhaust cluster capacity and observe pod vs node scaling.

Project 4 - Mini E2B/Replit sandbox runtime

Goal: learn control/data planes, scheduling and hostile-workload isolation.

Scope: 2-3 Linux/KVM hosts are enough.

Components:

sandbox API

Postgres lifecycle table

scheduler

host registry + heartbeats

host agent

Firecracker launcher

base snapshot/template

PTY/WebSocket gateway

session timeout/quota

metrics

Terraform owns: networks, control-plane VM/service, DB, object storage, sandbox hosts, IAM.

Runtime owns: individual microVM lifecycle.

Milestones:

1. Boot a microVM from API call.
2. Execute a command and return stdout.
3. Stream PTY over WebSocket.
4. Add CPU/RAM/session quotas.
5. Add snapshot cache.
6. Scheduler places by free memory + template locality.
7. Drain a host without accepting new sessions.
8. Kill a host and reconcile lost sessions.

Security review: guest has no cloud metadata credentials, bounded egress, jailed Firecracker, seccomp, cgroups, per-session filesystem and time limit.

Project 5 - Multi-region SRE + experimentation capstone

Goal: combine reliability, delivery, A/B testing and cost reasoning.

Product: use Project 2 or 3.

Architecture: primary region + warm standby/secondary region, global DNS, replicated/restore-able state, documented failover.

Tasks:

- define 99.9% SLO, RTO and RPO
- implement synthetic probes from two regions
- perform a regional failover game day
- measure actual recovery time/data loss
- deploy a new feature behind OpenFeature-compatible flagging
- run A/A first
- run A/B with stable user assignment and exposure events
- run a canary of the code version separately from the product experiment
- estimate cost/user, cost/job, egress and idle capacity

Architecture review output: write a 2-page ADR explaining why the system is or is not ready for active-active multi-cloud. A senior answer may correctly conclude that it is not worth it.

Part XVIII - Tool map and official reading links

Use official docs to learn semantics; use blog posts/videos after you know what the primitive is.

Foundations and infrastructure

- **Terraform:** [State](#), [Remote state](#), [Modules](#), [Dependency graph](#)
- **Ansible:** [Documentation](#)
- **Docker:** [Build best practices](#)
- **AWS Well-Architected:** [Framework](#)

Deployment and orchestration

- **GitHub Actions:** [OIDC cloud authentication](#)
- **Kubernetes:** [Deployments](#), [HPA](#), [Probes](#), [StatefulSet](#)
- **Argo CD:** [GitOps CD](#)
- **Argo Rollouts:** [Canary](#), [Blue/Green](#), [Experiments](#)
- **AWS ECS/Fargate:** [ECS developer guide](#)

Async and storage

- **Amazon S3:** [Presigned URLs](#)
- **Amazon SQS:** [Visibility timeout](#)
- **PostgreSQL:** [Official docs](#)
- **Redis:** [Official docs](#)
- **Apache Kafka:** [Documentation](#)

Observability and reliability

- **OpenTelemetry:** [Documentation](#), [Collector](#)
- **Prometheus:** [Documentation](#)
- **Grafana:** [Documentation](#)
- **Google SRE:** [SRE Workbook](#), [Error budgets](#)

Sandboxing / execution clouds

- **Firecracker:** [Design](#), [Jailer](#), [Snapshot support](#)
- **E2B:** [Product/runtime overview](#), [Security/BYOC](#)
- **Temporal:** [Documentation](#)

Feature flags and experiments

- **OpenFeature:** [Introduction/spec](#)
- **LaunchDarkly:** [Experimentation](#), [Designing experiments](#)

Public engineering case study

- **OpenAI:** [Scaling PostgreSQL to power 800 million ChatGPT users](#)

Part XIX - Final memory sheet

When somebody asks you to architect a system, do not begin with a cloud service name. Begin here:

WORKLOAD

What work happens? How much? How bursty? How long?

STATE

What must survive? What must be consistent? What can be cached?

BOUNDARIES

What is synchronous? What can queue? What must be isolated?

FAILURE

What can fail independently? What is the blast radius? RTO/RPO?

OWNERSHIP

Terraform? Host config? Runtime scheduler? App? GitOps controller?

FEEDBACK

Which metrics prove health? Which SLO controls rollout?

COST

What is the expensive resource per unit of customer value?

SIMPLICITY

What is the least complex system that meets the above?

And remember the progression:



Scale evolution

A VPS is not junior. Kubernetes is not senior. Multi-cloud is not advanced by definition. **The senior skill is matching architecture to constraints, knowing the failure modes before they happen, and keeping the system changeable and observable.**